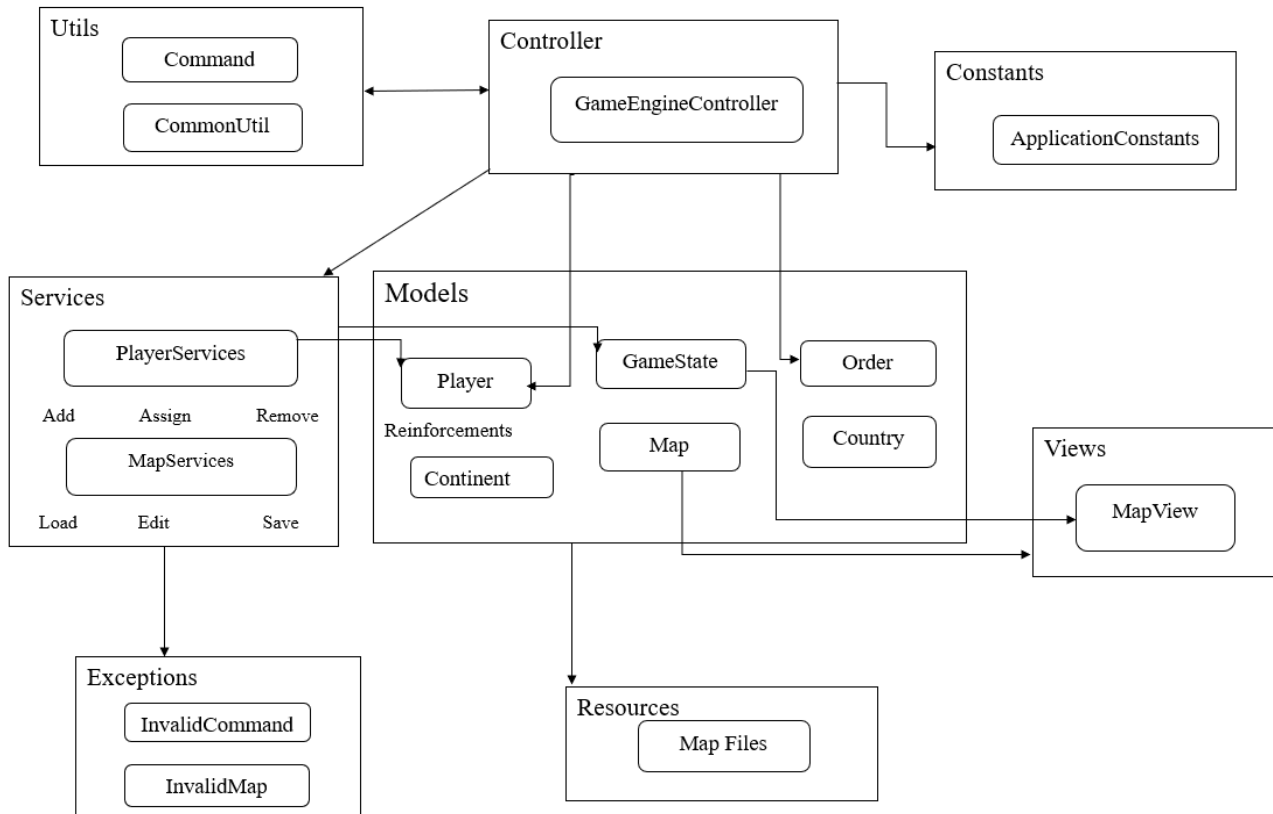


SOEN 6441 Advanced Programming Practices

(Team 09)

Build Version – 01 Architecture Diagram



1. Controller Package:

- GameEngineController**: This fundamental class orchestrates the primary command-handling logic within the system. It serves as the central hub for processing and executing commands initiated by users.

2. Services Package:

- MapServices**: This Class contains the core functionalities responsible for managing the state of the game map. It encloses tasks such as loading, editing, and saving map data, ensuring smooth and efficient map operations.
- PlayerServices**: This service layer facilitates operations related to players, including functionalities to add, remove, and assign continents, countries, and armies. It streamlines player-related tasks to enhance gameplay operations.

3. Constants Package:

- a. **ApplicationConstants:** This Component centralizes all static members and constants utilized throughout this application. By consolidating these values in one location, guarantees stability and simplicity for access among many modules and packages.

4. Utils Package:

- a. **Command:** This utility component is tasked with receiving and formatting commands issued by players. It ensures that commands are parsed correctly and ready for execution within the system.
- b. **CommonUtil:** Responsible for handling input files within the application. It streamlines the process of managing various file inputs, enhancing the overall efficiency of the system.

5. Models Package:

- a. **Player:** This Player class serves as a central management entity within this game, responsible for keeping track of various elements crucial to gameplay. This includes the continents and countries controlled by each player, the orders they issue during their turn, and the armies they possess on the map.
- b. **Order:** This Order class handles the execution of actions initiated by players during their turn. It specifies details such as the source and destination countries involved in the action and the number of armies to be moved between them.
- c. **GameState:** The GameState class manages the overall state of the game, including the Player and Order objects. It keeps track of all pending orders that have been issued by players but have not yet been executed, ensuring that the game progresses smoothly and according to the rules.
- d. **Map:** This Map class is responsible for managing the geographical layout of the game world, including countries and continents. It ensures that the map is properly connected, meaning that countries within the same continent are accessible to each other, and provides functionality to validate the map's accuracy.
- e. **Continent:** This Continent class manages the continents present in the game world, including assigning bonus values to them. Additionally, it establishes links with Country objects to designate which countries belong to each continent, facilitating gameplay mechanics related to continent control.
- f. **Country:** This Country class handles individual countries on the game map, keeping track of the number of armies present in each country and identifying their neighboring countries. This information is vital for strategic planning and resolving conflicts during gameplay.

6. View Package:

- a. **MapView:** This component showcases the current status of the game map when a user activates the "showmap" command during the gameplay phase. Additionally, it provides an overview of the entire map structure during the map editing phase.

7. Exceptions Package:

- a. **InvalidCommand:** This exception class type is triggered when a player inputs an invalid command. It provides clear and meaningful feedback to the player, guiding them on how to rectify the issue and continue with their gameplay.
- b. **InvalidMap:** This exception class is raised when an attempt is made to create an invalid game map. It delivers informative messages to players, helping them understand the nature of the error and take corrective actions if necessary.

8. Resources Package:

- a. **Map Files:** This resource repository manages all existing and newly created map files generated by users. It serves as a centralized storage location for map data, ensuring easy access and management of game maps within the system.